Due Date

You must *submit* the source code for the solution to this lab exercise to *Moodle* by

Wednesday, September 24, 2025

in order to receive full credit for this work. You must also *demonstrate* the solution to the instructor during class, at the earliest opportunity.

Lab Exercise 5a: Approximate π

In this exercise you are to write a simple Java program to calculate an approximate value for the mathematical constant **π**.

This lab exercise is also an opportunity to practice precise control over the output from your program.

Infinite Series to calculate an approximate value for π

An approximate value for the mathematical constant π can be calculated by the following infinite series:

$$\pi = 4\left(1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \frac{1}{11} + \cdots + \frac{(sign)}{2i-1}\right)$$

where $(sign)$ alternates between the values of $+1$ *and* -1.

The value i is a count of the terms in the series, starting at one.

Initial Programming Exercise

1. Write a program that prompts the user for a MAXIMUM value for i (**`loopMax`**).
2. Calculate the approximate value of π for $i = 1, 2, 3, \ldots,$ **`loopMax`**.
3. Debug your code for a small value like **`loopMax == 10`**.

Hint: Use a **`double`** variable called **`piOver4`**, and calculate an approximation of **`piOver4`**, using the following infinite series:

$$\texttt{piOver4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \frac{1}{11} + \cdots + \frac{(sign)}{2i-1}$$

Then it will be easy to calculate each approximation of **`pi`** itself: **`pi = piOver4 * 4;`**

It is important to understand that each **term** of the **piOver4** series is calculated by $\frac{(sign)}{2i-1}$. The process of adding up the terms results in the overall approximation. Our program lets the user choose *how many* terms to calculate.

Formatting the output

Use the `System.out.printf()` method to format the output in columns of text, as shown in the **Sample Output** section of this document. (We discussed how to write a **printf** format string in class.)

Controlling the Volume of the Output

After debugging your code for a small value like `loopMax==10`, change the code so it displays

- ALL results for the first 50 iterations,
- After the first 50 iterations of the loop, display the output less frequently: only when i is a multiple of 100, plus the very next iteration after the multiple of 100. In other words, we want output for all of the first 50 iterations, but after that we want a reporting interval of 100. Each "reporting interval" should report two cases: one where the approximation is too low (counter value is even), and the other where the counter value is too high (counter value is odd).
- Always display the FINAL iteration.

What this means is that, while the calculations must be done for ***every*** term of the series, our output will be less frequent overall, according to the rules stated above.

Optional Enhancement

If you have time, put the code that inputs `loopMax` inside a `try` block, and also implement a `catch` block to report an invalid input. The `try-catch` statements can be inside a loop, which keeps asking the user for integer input, until the user finally enters an integer value.

Sample Output

In the two output samples (which begin on the next page):

Count = the value of i (that is, the number of terms calculated so far).

Approximation = the approximate value of π for the current value of i.

Error = the difference between the approximation and the `Math.PI` constant:

Error = **Approximation** – `Math.PI`

Variance = the percent value of the error: **Variance** = (**Error** / `Math.PI`) * 100

termI = the i-th term of the `piOver4` calculation: $\frac{(sign)}{2i-1}$ where $(sign)$ is either `+1` *or* `-1`.

In the sample output (shown below and on the following page), text that the user types is shown in **BOLD** font. When the program actually runs, all text is shown in the same font.

Sample output for loopMax == 10

```
Enter maximum loop count:  10
Maximum loop count=10
 Count     Approximation   Error             Variance (%)      termI
     1:    4.0000000000   +00.8584073464   +27.3239544735 %  +1.0000000000
     2:    2.6666666667   -00.4749259869   -15.1173636843 %  -0.3333333333
     3:    3.4666666667   +00.3250740131   +10.3474272104 %  +0.2000000000
     4:    2.8952380952   -00.2463545584   -07.8417091430 %  -0.1428571429
     5:    3.3396825397   +00.1980898861   +06.3053969096 %  +0.1111111111
     6:    2.9760461760   -00.1655464775   -05.2695080425 %  -0.0909090909
     7:    3.2837384837   +00.1421458301   +04.5246423016 %  +0.0769230769
     8:    3.0170718171   -00.1245208365   -03.9636213300 %  -0.0666666667
     9:    3.2523659347   +00.1107732811   +03.5260230508 %  +0.0588235294
    10:    3.0418396189   -00.0997530347   -03.1752377109 %  -0.0526315789
Math.PI =  3.1415926536
```

Sample output for loopMax == 507

```
Enter maximum loop count:  507
Maximum loop count=507
 Count     Approximation   Error             Variance (%)      termI
     1:    4.0000000000   +00.8584073464   +27.3239544735 %  +1.0000000000
     2:    2.6666666667   -00.4749259869   -15.1173636843 %  -0.3333333333
     3:    3.4666666667   +00.3250740131   +10.3474272104 %  +0.2000000000
     4:    2.8952380952   -00.2463545584   -07.8417091430 %  -0.1428571429
     5:    3.3396825397   +00.1980898861   +06.3053969096 %  +0.1111111111
     6:    2.9760461760   -00.1655464775   -05.2695080425 %  -0.0909090909
     7:    3.2837384837   +00.1421458301   +04.5246423016 %  +0.0769230769
     8:    3.0170718171   -00.1245208365   -03.9636213300 %  -0.0666666667
     9:    3.2523659347   +00.1107732811   +03.5260230508 %  +0.0588235294
    10:    3.0418396189   -00.0997530347   -03.1752377109 %  -0.0526315789
    11:    3.2323158094   +00.0907231558   +02.8878077402 %  +0.0476190476
    12:    3.0584027659   -00.0831898877   -02.6480163673 %  -0.0434782609
    13:    3.2184027659   +00.0768101123   +02.4449418116 %  +0.0400000000
    14:    3.0702546178   -00.0713380358   -02.2707602059 %  -0.0370370370
    15:    3.2081856523   +00.0665929987   +02.1197209828 %  +0.0344827586
    16:    3.0791533942   -00.0624392594   -01.9875033551 %  -0.0322580645
    17:    3.2003655154   +00.0587728618   +01.8707982957 %  +0.0303030303
    18:    3.0860798011   -00.0555128525   -01.7670289750 %  -0.0285714286
    19:    3.1941879092   +00.0525952556   +01.6741589837 %  +0.0270270270
    20:    3.0916238067   -00.0499688469   -01.5905577976 %  -0.0256410256
    21:    3.1891847823   +00.0475921287   +01.5149045066 %  +0.0243902439
    22:    3.0961615265   -00.0454311271   -01.4461176905 %  -0.0232558140
    23:    3.1850504154   +00.0434577618   +01.3833035200 %  +0.0222222222
    24:    3.0999440324   -00.0416486212   -01.3257167879 %  -0.0212765957
```

Sample output for loopMax == 507

```
    25:    3.1815766854   +00.0399840318   +01.2727312626 %  +0.0204081633
    26:    3.1031453129   -00.0384473407   -01.2238168643 %  -0.0196078431
    27:    3.1786170110   +00.0370243574   +01.1785218993 %  +0.0188679245
    28:    3.1058897383   -00.0357029153   -01.1364590911 %  -0.0181818182
    29:    3.1760651769   +00.0344725233   +01.0972944961 %  +0.0175438596
    30:    3.1082685667   -00.0333240869   -01.0607386305 %  -0.0169491525
    31:    3.1738423372   +00.0322496836   +01.0265393116 %  +0.0163934426
    32:    3.1103502737   -00.0312423799   -00.9944758387 %  -0.0158730159
    33:    3.1718887352   +00.0302960816   +00.9643542301 %  +0.0153846154
    34:    3.1121872427   -00.0294054109   -00.9360032994 %  -0.0149253731
    35:    3.1701582572   +00.0285656036   +00.9092714032 %  +0.0144927536
    36:    3.1138202290   -00.0277724246   -00.8840237303 %  -0.0140845070
    37:    3.1686147496   +00.0270220960   +00.8601400296 %  +0.0136986301
    38:    3.1152814162   -00.0263112374   -00.8375126967 %  -0.0133333333
    39:    3.1672294682   +00.0256368146   +00.8160451536 %  +0.0129870130
    40:    3.1165965568   -00.0249960968   -00.7956504726 %  -0.0126582278
    41:    3.1659792728   +00.0243866193   +00.7762501999 %  +0.0123456790
    42:    3.1177865018   -00.0238061518   -00.7577733480 %  -0.0120481928
    43:    3.1648453253   +00.0232526717   +00.7401555282 %  +0.0117647059
    44:    3.1188683138   -00.0227243398   -00.7233382014 %  -0.0114942529
    45:    3.1638121340   +00.0222194804   +00.7072680286 %  +0.0112359551
    46:    3.1198560901   -00.0217365635   -00.6918963062 %  -0.0109890110
    47:    3.1628668428   +00.0212741892   +00.6771784731 %  +0.0107526882
    48:    3.1207615796   -00.0208310740   -00.6630736793 %  -0.0105263158
    49:    3.1619986930   +00.0204060394   +00.6495444081 %  +0.0103092784
    50:    3.1215946526   -00.0199980010   -00.6365561422 %  -0.0101010101
   100:    3.1315929036   -00.0099997500   -00.3183019294 %  -0.0050251256
   101:    3.1514934011   +00.0099007475   +00.3151505804 %  +0.0049751244
   200:    3.1365926848   -00.0049999688   -00.1591539484 %  -0.0025062657
   201:    3.1465677472   +00.0049750936   +00.1583621475 %  +0.0024937656
   300:    3.1382593295   -00.0033333241   -00.1061030007 %  -0.0016694491
   301:    3.1449149036   +00.0033222500   +00.1057505010 %  +0.0016638935
   400:    3.1390926575   -00.0024999961   -00.0795773472 %  -0.0012515645
   401:    3.1440864153   +00.0024937617   +00.0793789006 %  +0.0012484395
   500:    3.1395926556   -00.0019999980   -00.0636619136 %  -0.0010010010
   501:    3.1435886596   +00.0019960060   +00.0635348441 %  +0.0009990010
   507:    3.1435650383   +00.0019723847   +00.0627829540 %  +0.0009871668
Math.PI =  3.1415926536
```

Copyright © 2025 Peter Morgan. All rights reserved. You may **not** share this document with anyone or use it in any way other than as a participant in this course.